Apprentice — Verified Facts (Oct 13, 2026) and Build Docs: PRD.md, ARCHITECTURE.md, AGENTS.md + CLAUDE.md, SPEC.md

The settled plan holds up against the October 2026 docs: build Apprentice as one Cloud Run web app on `gemini-3.8-live` (GA 15 Sep 2026) and `gemini-3.8-flash` (GA 2 Sep 2026), with Model Armor as a standalone REST call, and keep Agent Runtime and Agent Gateway on the roadmap. The research does force **five changes** to the implementation details. The biggest is that the ephemeral token should carry the entire Live setup (SOP system instruction, tool, transcription, compression), because when a token has constraints and an empty field mask, the API ignores the browser's setup message. google github

TL;DR

- **Verified stack:** `gemini-3.8-live` handles real-time probing. It supports async (NON_BLOCKING) function calling by default, accepts 1 FPS JPEG frames plus 16 kHz PCM audio, and is free of charge on the free tier. `gemini-3.8-flash` (1M context, structured outputs) handles every batch call, agent arm and the judge. Files API keeps uploads for 48 h, up to 2 GB per file. Cloud Run caps HTTP/1 requests at 32 MiB, so recordings must go to GCS through signed URLs. Model Armor runs in asia-south1, works with any model, and the first 2M tokens/month are free.
- **Change these five things:** (1) mint ephemeral tokens with the full `liveConnectConstraints` setup, created server-side. (2) Enable `contextWindowCompression` plus `sessionResumption` on every Live session, because audio+video sessions die at **2 minutes** without compression. (3) Treat "Live speaks the probe" as a Day-1 risk: 3.8 Live has proactive audio permanently on, which may suppress unprompted speech. Ship a `sendClientContent` nudge and a PiP-text fallback. (4) Stamp deviation timestamps on the client, not from the model. (5) Use `spreadsheets.readonly` (sensitive, fine for test users) for the hackathon and name `drive.file` + Picker (non-sensitive) as the production scope.
- **Defer Agent Runtime/Agent Gateway; it would be a force-fit.** Agent Gateway (preview) governs MCP/A2A traffic between agent fleets, and Agent Runtime is a managed agent host. Apprentice has two single-shot Gemini calls, no tools, no agent-to-agent traffic, and must run on Cloud Run anyway. The four files below are ready to commit.

Key Findings (verified facts and what they change)

#	Area	Verified fact (Oct 2026)	Impact on Apprentice
1	Live model	`gemini-3.8-live` is "the default option for most low-latency voice agent experiences". Inputs are text, images, audio and video; outputs are text and audio. google Input limit is 131,072 tokens. Structured outputs are **not** supported. google google	Use it. The tool call is the only structured channel out of Live.
2	Live function calling	On 3.8 Live, async calling is the default (`NON_BLOCKING`). `BLOCKING` is opt-in. Scheduling `SILENT` / `WHEN_IDLE` / `INTERRUPT` is supported. 3.1 Flash Live Preview is sync-only. google datacamp	Declare `log_deviation` NON_BLOCKING and reply with `scheduling: SILENT` so the acknowledgement doesn't trigger extra speech.
3	Live media	Video frames are max 1 FPS JPEG; audio is raw 16-bit PCM at 16 kHz in and 24 kHz out. google google Standard input resolution is 768×768. "Video frames are sent to the model by default, so only send frames when needed". google +2	Downscale frames to ⩽768 px on the long edge and send JPEG at 1 FPS. Pause frames when the tab is idle.
4	Live session limits	Without compression, audio-only sessions last 15 min and audio+video sessions **2 min**. A connection lives about 10 min. Resumption tokens stay valid for 2 h. Before closing, the server sends `GoAway` with `timeLeft`. google google	Compression is mandatory even for 3–4 min sessions. Implement the resumption handle and handle GoAway.
5	Proactive audio	3.8 Live has proactive audio "permanently enabled". The model "generat[es] audio responses only when addressed by the user". google google	**Risk to the spoken-probe beat.** Mitigate with a client `sendClientContent` nudge and PiP text. Kill switch on Day 1.
6	Ephemeral tokens	Created with `client.auth_tokens.create`. Defaults: `newSessionExpireTime` 1 min, `expireTime` 30 min (max <20 h), `uses` 1 (resuming doesn't count as a use). The token can be locked with `liveConnectConstraints`. With constraints	The backend builds the full Live config (SOP, tool, transcription, compression, resumption) into the token. The browser only sends media. This also blocks tool injection. vpncentral

#	Area	Verified fact (Oct 2026)	Impact on Apprentice
		and an empty field mask, the setup comes "entirely" from the token and the browser setup message "is ignored". Still Preview. google +3	
7	Live pricing	Free tier: free of charge. Paid: audio in $0.005/min, audio out $0.018/min, image/video in $0.002/min. google zentor	Demo cost is negligible. Use a paid key (Tier 1) anyway, because free-tier data is "used to improve our products" and limits are lower.
8	Batch model	`gemini-3.8-flash` is GA (2 Sep 2026): 1,048,576 input / 65,536 output tokens, structured outputs, thinking `low/medium/high` (`minimal` errors). Introductory price is $0.75/$3.75 per 1M tokens. google No 3.8 Pro exists; the newest Pro is `gemini-3.1-pro-preview` (preview). google +2	Flash for everything. Pro is optional and only for the graph builder, with a preview-model caveat.
9	Files API	"per-file maximum size of 2 GB. Files are stored for 48 hours", with 20 GB per project, at no cost. google `video/webm` is supported. Timestamps use `MM:SS`, default sampling is 1 FPS, and `fps` / `media_resolution` are tunable. google educative	Re-upload from GCS when a Files URI is older than 48 h. Keep originals in GCS.
10	Cloud Run	Max HTTP/1 request is "32 MiB if using HTTP/1 server. No limit if using HTTP/2 server". Max request timeout is 60 min. google	Browser → GCS via a V4 signed URL. dev Processing runs as a background task on an instance with CPU always allocated.
11	Model Armor	Works with any model via REST ("protects all LLMs"). Free up to 2M tokens/month. Needs the `roles/modelarmor.user` role and a regional endpoint (the global endpoint can't sanitize). asia-south1 (Mumbai) is available but has "limited multi-language detection" (non-English → `Skipped`). google +2	Use asia-south1 with English content. JA/HI on the roadmap needs a different region.

#	Area	Verified fact (Oct 2026)	Impact on Apprentice
12	Agent Runtime / Gateway	Agent Runtime is a "fully-managed, opinionated runtime" for agents. Agent Gateway (preview) "manages your agent fleet from one control point", and Model Armor plugs in for MCP/A2A traffic. google google	Defer to the "org-wide agent fleet" roadmap slide.
13	OAuth	`spreadsheets.readonly` is **sensitive**, `drive.file` is **non-sensitive**, and `drive.readonly` and Gmail read scopes are **restricted**. Unverified apps are capped at 100 new users and show a warning screen. google google	Testing mode with the demo account as a test user plus `spreadsheets.readonly`. No Gmail scopes.
14	Document PiP	Chrome 116+. `requestWindow()` "requires transient activation". The window "floats on top of other windows", "never outlives the opening window", and its position can't be set. Resize needs a user gesture. chrome +2	The Start button lives inside the PiP window, so the click there gives the activation for `getDisplayMedia`.
15	Coding agents	Pi loads `AGENTS.md` (or `CLAUDE.md`) from `~/.pi/agent/`, from parent directories and from cwd, and concatenates them. Claude Code reads CLAUDE.md, and `@AGENTS.md` inside CLAUDE.md imports it. Claude Code reads AGENTS.md directly only when no CLAUDE.md exists (v2.1.277+). npmjs +2	Root `AGENTS.md` is the source of truth. `CLAUDE.md` = `@AGENTS.md` plus Claude-only notes.

Decisions the research says to revise

1. **Live config lives in the token, not the browser** (finding 6).
2. **Compression + resumption are required, not optional** (finding 4).
3. **Spoken probe has a 3-tier fallback:** model speech → client nudge → PiP text plus browser `speechSynthesis` (finding 5).
4. **Deviation timestamp = client receipt time**, because Live has no recording clock. The model's own timestamp is advisory.
5. **Run each exam case 3× per arm** and report the majority decision plus its stability. It costs ~60 Flash calls and makes the "pilot" chart defensible.

6. **Don't claim malicious-URL detection on fake URLs.** Model Armor's URL filter matches known-bad URLs, so synthetic `.test` links won't trip it. The demo beat is the prompt-injection/jailbreak filter.

File 1 — PRD.md

markdown

```
# Apprentice — Product Requirements Document
Version 1.0 · Oct 13, 2026 · Google Cloud AI Builder Cup 2026 · Category: Future o
Team: Person A (ML/pipeline/Live backend), Person B (UI/deploy/deck/video)

## 1. Problem
When a senior operator leaves, the SOP stays but the judgment goes with them. SOPs

JAPAC urgency (directional; cite carefully in the deck):
- Japan: METI's DX Report (Sep 2018) warned of a "2025 digital cliff": up to ¥12 t
- India: Aon's 32nd Annual Salary Increase and Turnover Survey puts overall attrit
- 42% of institutional knowledge is unique to the individual employee, so coworker

## 2. Solution (one-liner)
An AI apprentice watches an expert work in Google apps (screen + voice) and asks t

## 3. Users and personas
| Persona | Goal | Primary surfaces |
|---|---|---|
| Expert (AP lead, 8 yrs) | Capture judgment without writing documentation; answer
| Successor (new AP analyst) | Learn exceptions and their reasons; see evidence cl
| Team lead / Ops manager | Know whether knowledge transferred before the expert l
| Judge (demo mode) | Understand the product in <3 minutes without login | Demo mo

## 4. Job to be done
"When a critical expert is about to leave or rotate, help me capture the decisions

## 5. Positioning vs the commodity layer
| Layer | Examples | Captures | Apprentice stance |
|---|---|---|---|
| Enterprise search / assistants | Gemini Enterprise, Glean | Documents, WHAT exis
| Step recorders | Scribe, Tango | Clicks, WHAT was done | Not competing; we recor
| Policy KB / RAG | Drive/Confluence + vector DB | Written rules | Commodity; roac
| **Apprentice** | — | **WHY + exceptions + verified transfer** | Differentiator:

## 6. Form-factor decision: web app + Document Picture-in-Picture
| Criterion | Web app (chosen) | Desktop app | Chrome extension |
|---|---|---|---|
| Judge access | Open a deployed Cloud Run URL | Download/install; no clean deploy
| 3-day feasibility | High | Packaging/signing overhead | Chrome Web Store review
| Capture in Sheets/Docs | getDisplayMedia video works on any app | Works | DOM ca
| Future apps (SAP, ServiceNow) | Any web app via screen capture | Any app | Per-s
```

| Always-on-top companion | Document PiP (Chrome 116+) | Native | Side panel (tab-
| Deployment requirement (Cloud Run/Firebase) | Native fit | Backend only | Backen
Decision: web app. The companion window uses `documentPictureInPicture.requestWind

7. Goals and non-goals
Goals (hackathon):
G1. Record expert sessions (screen + mic) and process them into segments, SOP devi
G2. Ask CDM probes live (voice + PiP text) and in batch; store answers.
G3. Build a decision graph: cue → goal → options → rule → exception-to-SOP-rule, e
G4. Run a successor exam: SOP-only arm vs SOP+graph arm on 10 cases, with an LLM j
G5. Screen all untrusted text with Model Armor; flag the planted injection email.
G6. Demo mode works without login.
Non-goals: human-successor arm; any writes to Sheets/Gmail; computer use / autonom

8. User stories and acceptance criteria
US1 Expert records a session. AC: one click in the PiP window starts tab capture +
US2 Expert gets a live probe. AC: within 15 s of skipping PO-match on a consignmen
US3 Expert answers batch probes. AC: ≤2 min after upload, batch probes appear in P
US4 Team lead sees discovered variants. AC: after ≥3 sessions are processed, the D
US5 Successor explores the graph. AC: graph renders ≥1 exception node per planted
US6 Safety. AC: the injection email text is flagged `pi_and_jailbreak` MATCH_FOUND
US7 Exam. AC: 10 cases (2 held-out, 8 generated incl. ≥3 traps) are expert-labelle
US8 Judge demo mode. AC: `/demo` loads without login in <3 s and shows seeded sess
US9 Connect Sheet (demo expert only). AC: incremental consent for `spreadsheets.re

9. Functional requirements by module
Capture (B): getDisplayMedia (prefer browser tab, exclude self), mic via getUserMe
Live Probe (A backend, B client): backend mints a constrained ephemeral token cont
Batch Discovery (A): Files API upload; segmenter; deviation detector (SOP + sheet
Probe & Graph (A pipeline, B UI): CDM probe generator; answer capture; graph build
Exam & Eval (A pipeline, B report): scenario generator with traps; expert labellin
Model Armor (A): screen segment text/transcripts and probe answers before graph bu
Auth / Connect Sheet (B UI, A endpoint): Firebase Auth Google (openid/email/profil
Demo mode (B, seeded by A): read-only workspace `demo` seeded from a real run.

10. Success metrics
Metric	Target	Notes
Graph-arm decision agreement with expert labels	≥80% (≥8/10 cases, majority of
Graph arm − SOP-only arm agreement	≥30 points	Primary; finalize thresholds f
Trap handling (graph arm)	≥2 of 3 traps correct	Shows no over-generalization
Planted-exception recall in batch deviations	3/3 types detected across trainin

```
| Live probe latency | ≤15 s from deviation to PiP text | UX |
| Injection email flagged | 100% (1/1) | Safety beat |
| Judge leniency check | 3 hand-checked cases agree with judge within 1 point | Ev
All results are labelled "pilot (n=10, synthetic data, one expert)".

## 11. Privacy and compliance
- Synthetic data only: fake vendors, fake GSTINs, `.test`/`example` domains; a dec
- Consent: a recording banner and explicit consent checkbox before first capture;
- Japan APPI and India DPDP Act 2023 alignment (roadmap for production): purpose l
- Redaction: Model Armor Sensitive Data Protection screens text before it enters t
- Retention: Gemini Files are auto-deleted after 48 h; GCS originals deleted after

## 12. Risks, fallbacks, kill switches
| Risk | Likelihood | Fallback / kill switch |
|---|---|---|
| Live doesn't call `log_deviation` reliably | Med | End of Day 1 (Oct 14, 22:00 I
| Live won't speak the probe (proactive audio) | Med | Client `sendClientContent`
| Ephemeral token/SDK version mismatch (v1alpha vs v1beta) | Med | Pin SDK version
| Deviation detector misses planted exceptions | Low–Med | Sheet diff as ground tr
| Graph arm doesn't beat baseline | Low–Med | Inspect misses → new probes → re-ans
| Model Armor latency/region issue | Low | Fall back to us-central1 template; cach
| Cloud Run processing timeout | Low | Background task + CPU always allocated; Clo
| Recording quality (Sheets text unreadable) | Low | Zoom Sheets to 125%; media_re
Cut order if behind: Live → Connect Sheet → probe depth → scenario count. Never cu

## 13. Demo script (target 2:45, hard cap 2:59)
| Time | Beat | On screen |
|---|---|---|
| 0:00–0:15 | Hook | "42% of an employee's job knowledge is unique to them" (Panop
| 0:15–0:45 | Expert works | Sheets invoice queue + Gmail; approves consignment in
| 0:45–1:05 | Live probe | PiP pins "Why did you skip the PO match for Kaveri?"; L
| 1:05–1:25 | Discovery | Variants across 6 sessions; deviations linked to SOP §3.
| 1:25–1:45 | Graph | New exception node "Consignment vendor → reconcile against m
| 1:45–2:00 | Safety | Injection email flagged by Model Armor; excluded from graph
| 2:00–2:25 | Exam | Same case: SOP-only arm rejects/holds wrongly; graph arm corr
| 2:25–2:35 | Results | Pilot bar chart; a graph-arm miss → new probe to the exper
| 2:35–2:45 | Close | "Apprentice captures the why — and proves it transferred." U

## 14. Roadmap
1. Chrome extension: Gmail DOM context, side panel, per-site adapters.
2. Policy KB from Drive/Confluence (RAG) to auto-ground the SOP.
3. Multi-language Live interviews (Japanese, Hindi); Gemini 3.8 Live supports many
```

```
4. Org-wide agent fleet: compiled SOP+graph agents deployed on Agent Runtime, gove
5. Enterprise path: Gemini Enterprise Agent Platform (formerly Vertex AI) with reg
6. Human-successor exam arm; spaced-repetition training for successors.

## 15. Judging-criteria mapping
| Criterion (weight) | Evidence in Apprentice |
|---|---|
| Technical Merit & GenAI (40%) | Gemini 3.8 Live with async tool calling + epheme
| Problem Alignment & Impact (25%) | JAPAC knowledge-loss story; AP approval is un
| Innovation & Creativity (25%) | CDM probes from an AI apprentice; "prove it tran
| UX & Solution Design (10%) | Always-on-top PiP companion; voice answers; clip-li
```

File 2 — ARCHITECTURE.md

markdown

````
# Apprentice — Architecture
Solid = MVP. Dashed = stretch/roadmap.

## 1. System diagram
```mermaid
flowchart LR
  subgraph Browser["Browser (Chrome 116+)"]
    REC[Recorder<br/>getDisplayMedia + mic<br/>MediaRecorder webm]
    PIP[PiP companion<br/>documentPictureInPicture]
    LIVEC[Live client<br/>@google/genai live.connect]
    PUI[Probe answer UI]
    GV[Graph view<br/>Cytoscape]
    EX[Exam report + pilot chart]
    DM[Demo mode /demo]
    GIS[GIS token client<br/>spreadsheets.readonly]
  end

  subgraph CR["Cloud Run: apprentice (FastAPI, asia-south1)"]
    API[REST /api/*]
    TOK[live_token]
    UP[uploads: V4 signed URL]
    PIPE[pipeline: segment / deviations / variants / probes / graph]
    EXAM[exam: scenarios / arms / judge / report]
    MA[moderation client]
    STATIC[static React build]
  end

  FS[(Firestore<br/>workspaces/demo, workspaces/live)]
  GCS[(Cloud Storage<br/>recordings)]
  SM[(Secret Manager<br/>GEMINI_API_KEY)]
  FA[Firebase Auth]
  GLIVE[Gemini Live API<br/>gemini-3.8-live]
  GFILES[Gemini Files API]
  GFLASH[gemini-3.8-flash]
  MARMOR[Model Armor<br/>asia-south1 template]
  SHEETS[Sheets API v4 read-only]

  Browser -- ID token --> API
  FA -. sign-in .- Browser
  TOK -- auth_tokens.create --> GLIVE
  LIVEC == ephemeral token, WSS: frames 1 FPS + PCM16 ==> GLIVE
````
````

```
  GLIVE -- tool call log_deviation + transcripts --> LIVEC
  LIVEC -- POST deviation / answer --> API
  REC -- PUT webm via signed URL --> GCS
  UP --> GCS
  PIPE --> GFILES --> GFLASH
  PIPE --> MA --> MARMOR
  EXAM --> GFLASH
  EXAM --> MA
  API <--> FS
  CR -- secret env --> SM
  GIS -- values.get --> SHEETS
  GIS -- snapshot rows --> API
  STATIC --> Browser
  AR[Agent Runtime]:::road
  AG[Agent Gateway]:::road
  EXAM -.-> AR -.-> AG
  classDef road stroke-dasharray: 5 5,color:#888;
````

2. Pipeline flowchart

````mermaid
flowchart TD
  A[Session uploaded to GCS] --> B[Upload to Gemini Files API; poll until ACTIVE]
  B --> C[Segmenter: MM:SS segments, app, action, invoice, decision, transcript]
  C --> MA1{Model Armor<br/>segment text}
  MA1 -- flagged --> Q[Quarantine: mark segment, excluded from graph]
  MA1 -- clean --> D[Deviation detector: SOP + segments + sheet diff + live deviat
  D --> E[Dedup vs live deviations: same sop_rule and |dt| ≤ 20 s]
  E --> F{≥3 sessions processed?}
  F -- yes --> G[Variant discovery: ONE call over all sessions' segments + deviati
  F -- no --> H
  G --> H[CDM probe generator: ≤4 probes per new deviation/variant]
  H --> I[Expert answers: voice (Live) or typed]
  I --> MA2{Model Armor<br/>answer text}
  MA2 -- clean --> J[Graph builder: nodes/edges citing clips + SOP §]
  MA2 -- flagged --> Q
  J --> K[(Firestore graph)]
  X[Sheet diff before/after]:::opt -.-> D
  classDef opt stroke-dasharray: 5 5;
````

3. Live sequence

````mermaid
sequenceDiagram
  autonumber
  participant U as Expert (PiP)
  participant B as Browser app
  participant API as Cloud Run API
  participant SM as Secret Manager
  participant L as Gemini Live (gemini-3.8-live)
  participant FS as Firestore
  U->>B: Click Start (inside PiP window)
  B->>API: POST /api/live/token {session_id} (Firebase ID token)
  API->>SM: GEMINI_API_KEY (cached at startup)
  API->>L: auth_tokens.create(uses=1, expire 30m, new_session 1m, liveConnectConst
  L-->>API: token.name
  API-->>B: {token, model, expires_at}
  B->>L: live.connect(apiKey=token) (WSS BidiGenerateContentConstrained)
  loop every 1 s
    B->>L: sendRealtimeInput video JPEG (≤768 px)
  end
  loop every 20–40 ms
    B->>L: sendRealtimeInput audio PCM16 16 kHz (mic)
  end
  L-->>B: toolCall log_deviation{sop_rule, question, invoice_id, timestamp?}
  B->>B: t_client = now - rec_start - 3 s
  B->>API: POST /api/sessions/{id}/deviations (source=live)
  API->>FS: deviations + probes (source=live)
  B->>L: toolResponse {id, name, response{ok, probe_id}, scheduling: SILENT}
  B->>U: PiP pins question text
  L-->>B: audio "Why did you skip the PO match?" (+ outputTranscription)
  alt no audio within 4 s
    B->>L: sendClientContent(user: "Ask the expert now, one sentence: <q>", turnCo
    alt still silent after 4 s
      B->>U: speechSynthesis.speak(q)
    end
  end
  U->>L: voice answer (mic)
  L-->>B: inputTranscription chunks
  B->>API: POST /api/probes/{pid}/answer {text, source: voice_live}
  API->>FS: probe_answers (after Model Armor)
  L-->>B: sessionResumptionUpdate {newHandle} (stored in memory)
  L-->>B: goAway {timeLeft}
  B->>API: POST /api/live/token (resume)
````

```
  B->>L: reconnect with handle
```

4. Batch sequence
````mermaid
sequenceDiagram
  autonumber
  participant B as Browser
  participant API as Cloud Run
  participant GCS as Cloud Storage
  participant F as Gemini Files API
  participant G as gemini-3.8-flash
  participant MA as Model Armor
  participant FS as Firestore
  B->>API: POST /api/sessions/{id}/upload-url
  API-->>B: V4 signed PUT URL (15 min, content-type video/webm)
  B->>GCS: PUT recording.webm
  B->>API: POST /api/sessions/{id}/finalize
  API->>FS: status=uploaded
  B->>API: POST /api/sessions/{id}/process
  API-->>B: 202 Accepted (background task)
  API->>GCS: download blob
  API->>F: files.upload; poll state until ACTIVE
  API->>G: segmenter(video, SOP) → SegmentList
  API->>MA: sanitizeUserPrompt(segment text)
  API->>G: deviation detector(SOP, segments, sheet diff, live deviations) → Deviat
  API->>G: probe generator → ProbeList
  API->>FS: segments, deviations, probes; status=processed
  B->>API: GET /api/sessions/{id} (poll every 3 s)
````

5. Exam flow (no label leakage)
````mermaid
flowchart TD
  S[Scenario generator: 8 cases from graph + SOP + trap spec] --> C[Exam cases]
  H[2 held-out recordings → cases] --> C
  C --> L[Expert labels each case: decision + 1-line rationale]
  L -. stored in exam_cases only .-> LBL[(labels)]
  C --> MA1{Model Armor: case inputs}
  MA1 --> A1[Arm A: SOP only ×3]
  MA1 --> A2[Arm B: SOP + graph ×3]
  A1 --> MA2{Model Armor: outputs}
```

```
  A2 --> MA2
  MA2 --> J[Judge gemini-3.8-flash: rubric vs label]
  LBL --> J
  J --> R[Report: per-case table + pilot bar chart]
  R --> M{Graph-arm miss?}
  M -- yes --> P[New probe to expert: asks about the situation, NOT the label]
  P --> ANS[Expert answer] --> GB[Graph rebuild]
  GB -. never reads .-x LBL
````

6. Roadmap
````mermaid
flowchart LR
  MVP[MVP: web app + PiP + Live + graph + exam]
  MVP -.-> EXT[Chrome extension: Gmail DOM, side panel]:::r
  MVP -.-> KB[Policy KB from Drive/Confluence]:::r
  MVP -.-> ML[Live interviews in JA/HI]:::r
  MVP -.-> FLEET[Compiled agents on Agent Runtime]:::r
  FLEET -.-> GW[Agent Gateway + Model Armor inline]:::r
  MVP -.-> VTX[Agent Platform / Vertex regional path, CMEK, VPC-SC]:::r
  classDef r stroke-dasharray: 5 5,color:#666;
````

7. Component notes
- **One Cloud Run service** in asia-south1: FastAPI serves `/api/*` and the Vite b
- **Background processing**: `asyncio.create_task` after returning 202. CPU stays
- **Secrets**: `GEMINI_API_KEY` mounted from Secret Manager (`--set-secrets`). It
- **Signed URLs on Cloud Run**: V4 signing through IAM `signBlob`. The runtime ser
- **Model Armor**: regional endpoint `modelarmor.asia-south1.rep.googleapis.com`,
- **Agent Runtime / Agent Gateway**: not used. Rationale: the competition requires

---

File 3 — AGENTS.md

markdown

````
# AGENTS.md — Apprentice (read this before writing any code)

Tool-neutral instructions for AI coding agents (Claude Code via CLAUDE.md import;

## Project summary
Apprentice watches an expert work in Google Sheets + Gmail (screen + voice), asks

## Repo layout
```
/AGENTS.md  /CLAUDE.md  /README.md  /Dockerfile  /Makefile
/backend/            # Person A owns (FastAPI, Python 3.12)
  app/main.py        # FastAPI app, static mount, routers
  app/contract.py    # Pydantic v2 models = SOURCE OF TRUTH
  app/routers/       # auth.py live.py uploads.py sessions.py probes.py graph.py e
  app/pipeline/      # files.py segment.py deviations.py variants.py probes.py gra
  app/exam/          # scenarios.py arms.py judge.py report.py
  app/safety/armor.py
  app/store.py       # Firestore access (workspace-scoped)
  app/gemini.py      # client, model IDs, structured-call helper
  tests/
/frontend/           # Person B owns (Vite + React + TS)
  src/types/contract.ts   # GENERATED from backend contract — do not hand-edit
  src/api/  src/pip/  src/recorder/  src/live/  src/views/  src/demo/
/shared/schema.json  # generated JSON Schema of the contract
/fixtures/           # JSON fixtures matching the contract (demo seed + UI dev)
/prompts/            # one .md file per Gemini call, versioned
/scripts/            # export_schema.py gen_types.sh seed_demo.py make_synthetic.p
/docs/               # PRD.md ARCHITECTURE.md SPEC.md
/eval/               # exam outputs, chart PNG, hand-check notes
```

## Tech stack (pin these)
- Python 3.12, `uv`; FastAPI, uvicorn, pydantic v2, `google-genai` (latest 2026-10
- Node 22, pnpm; Vite, React 19, TypeScript strict, `@google/genai` (latest 2026-1
- Models (env-overridable): `LIVE_MODEL=gemini-3.8-live`, `FLASH_MODEL=gemini-3.8-
- GCP: project `apprentice-aibc`, region `asia-south1`; Cloud Run service `apprent

## Commands
```
make setup        # uv sync; cd frontend && pnpm i
make dev          # backend :8080 (uvicorn --reload) + vite :5173 (proxy /api → 80
````
````

```
make types        # uv run python scripts/export_schema.py > shared/schema.json &&
make test         # uv run pytest -q backend/tests && pnpm --dir frontend test --r
make lint         # uv run ruff check backend && uv run ruff format --check backen
make build        # pnpm --dir frontend build → backend/app/static
make seed-demo    # uv run python scripts/seed_demo.py fixtures/demo_seed.json
make deploy       # scripts/deploy.sh (gcloud run deploy --source .)
```
Deploy flags (scripts/deploy.sh): `gcloud run deploy apprentice --source . --regio

Coding conventions
- Python: full type hints; Pydantic v2 models only (`model_config = ConfigDict(ext
- TS: `strict: true`; React function components + hooks only; no `any` (use genera
- Every Gemini batch call goes through `app/gemini.py::structured(model, prompt_id

Data-contract-first rule
1. `backend/app/contract.py` is the source of truth. Change it first, then run `ma
2. Never hand-edit `frontend/src/types/contract.ts`.
3. Fixtures in `/fixtures` must validate against the contract (`pytest backend/tes
4. API shapes are in docs/SPEC.md §3; changing a shape needs a message to the othe

Ownership boundaries
- Person A (Claude Code): `backend/app/pipeline`, `backend/app/exam`, `backend/app
- Person B (Pi): `frontend/**`, `Dockerfile`, `scripts/deploy.sh`, deck, video.
- Shared (announce before editing): `backend/app/contract.py`, `backend/app/main.p
- Agents must not edit files outside their owner's area unless the task says so.

Guardrails (hard rules)
- Never commit secrets, `.env`, service-account keys, OAuth client secrets, or tok
- The Gemini API key exists only server-side (Secret Manager → env). The browser c
- No tokens (ephemeral, OAuth access, Firebase) in localStorage/sessionStorage/Ind
- All Gemini batch outputs are structured through Pydantic schemas. No free-text p
- All untrusted text (segment text/transcripts, email text seen in video, probe an
- Exam labels (`expert_label`) are NEVER passed to the graph builder, probe genera
- No writes to Google Sheets or Gmail. Sheets access is read-only and only for the
- Demo mode (`/demo`, workspace `demo`) must work at every commit; CI-like check:
- Deploy at least once a day (end of day IST) and post the URL in the team chat.
- Do not add new cloud services, databases, vector stores or frameworks without bo

Prompts
- One file per call in `/prompts`: `segmenter.md, deviations.md, variants.md, prob
- Front-matter: `id`, `version` (int), `model`, `thinking_level`, `output_schema`
- Use `{{SOP}}`, `{{SEGMENTS}}`-style placeholders filled by `app/prompts.py`.

```
## Fixtures
- `fixtures/*.json` mirror API responses (session list, session detail, probes, gr
- `fixtures/demo_seed.json` is produced from a real run by `scripts/export_demo.py

## Testing expectations (minimal but real)
- Backend: contract round-trip tests; fixture validation; `test_no_label_leakage`
- Frontend: type-check; one render test per view against fixtures; manual checklis

## Definition of done (per task)
Code merged to `main`; `make lint test` green; types regenerated if the contract c

## Do NOT build
Human-successor arm; Sheets/Gmail writes; computer use; Chrome extension; multi-la
```

CLAUDE.md

markdown

````
@AGENTS.md

## Claude Code only (Person A)
- Your scope: backend/app/pipeline, backend/app/exam, backend/app/safety, backend/
- Use plan mode before changing backend/app/contract.py; then run `make types`.
- Prefer running `uv run pytest -q backend/tests` after each change.
````
````

> Note: in Claude Code the `@AGENTS.md` line imports the file. Because a CLAUDE.md

---

## File 4 — SPEC.md

````markdown
# Apprentice — Implementation Spec (v1.0, Oct 13 2026)

## 0. Constants
```
LIVE_MODEL = "gemini-3.8-live"
FLASH_MODEL = "gemini-3.8-flash"      # segment, deviations, variants, probes, gra
PRO_MODEL = "gemini-3.1-pro-preview"  # optional, graph builder only if Flash qual
REGION = "asia-south1"
WORKSPACES = {"demo", "live"}
LIVE_FRAME_FPS = 1; LIVE_FRAME_MAX_EDGE = 768; LIVE_JPEG_QUALITY = 0.7
LIVE_AUDIO_RATE = 16000; LIVE_AUDIO_CHUNK_MS = 40
PROBE_MIN_INTERVAL_S = 45; PROBE_MAX_PER_SESSION = 3
DEDUP_WINDOW_S = 20
EXAM_RUNS_PER_ARM = 3
```

## 1. Data contract (backend/app/contract.py)
```python
from __future__ import annotations
from datetime import datetime
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class M(BaseModel):
    model_config = ConfigDict(extra="forbid")
````

```python
Timestamp = str  # "MM:SS" (H:MM:SS if > 1h)
Decision = Literal["approve", "reject", "hold", "escalate"]
Arm = Literal["sop_only", "sop_graph"]

class ClipRef(M):
    session_id: str
    start_ts: Timestamp
    end_ts: Timestamp

class SOPSection(M):
    id: str            # "§3.2"
    title: str
    text: str

class SOP(M):
    id: str
    title: str
    version: int
    sections: list[SOPSection]
    full_text: str

class ModerationResult(M):
    id: str
    target_kind: Literal["segment", "probe_answer", "exam_input", "agent_output",
    target_id: str
    template: str
    match_found: bool
    filters: dict[str, str]          # e.g. {"pi_and_jailbreak": "MATCH_FOUND", "s
    skipped: bool = False
    blocked: bool
    checked_at: datetime

class Session(M):
    id: str
    workspace: Literal["demo", "live"]
    expert_uid: str
    kind: Literal["training", "heldout"]
    status: Literal["created", "uploading", "uploaded", "processing", "processed",
    gcs_uri: str | None = None
    gemini_file_name: str | None = None
    gemini_file_uploaded_at: datetime | None = None
```

```
    duration_s: int | None = None
    started_at: datetime
    ended_at: datetime | None = None
    live_enabled: bool = True
    sheet_snapshot_before_id: str | None = None
    sheet_snapshot_after_id: str | None = None
    error: str | None = None

class Segment(M):
    id: str
    session_id: str
    start_ts: Timestamp
    end_ts: Timestamp
    app: Literal["sheets", "gmail", "other"]
    action: str
    invoice_id: str | None = None
    vendor: str | None = None
    decision: Decision | Literal["none"] = "none"
    transcript_excerpt: str = ""
    on_screen_text: str = ""
    moderation_id: str | None = None
    quarantined: bool = False

class Deviation(M):
    id: str
    session_id: str
    source: Literal["live", "batch"]
    sop_rule: str                    # "§3.2"
    timestamp: Timestamp
    t_seconds: int
    invoice_id: str | None = None
    description: str
    question: str
    confidence: float = Field(ge=0, le=1)
    dedup_of: str | None = None      # id of the live deviation it duplicates

class Variant(M):
    id: str
    name: str
    description: str
    sop_rules: list[str]
    session_ids: list[str]
```

```python
    deviation_ids: list[str]
    exemplar_clips: list[ClipRef]
    frequency: int

class Probe(M):
    id: str
    session_id: str | None = None
    deviation_id: str | None = None
    variant_id: str | None = None
    source: Literal["live", "batch", "exam_miss"]
    cdm_type: Literal["cue", "goal", "option", "experience", "rule", "anomaly", "h
    question: str
    sop_rule: str | None = None
    clip: ClipRef | None = None
    status: Literal["open", "answered", "skipped"] = "open"
    created_at: datetime

class ProbeAnswer(M):
    probe_id: str
    text: str
    source: Literal["voice_live", "typed", "edited"]
    answered_at: datetime
    moderation: ModerationResult | None = None

class DecisionNode(M):
    id: str
    kind: Literal["cue", "goal", "option", "rule", "exception"]
    label: str
    detail: str
    sop_rule: str | None = None      # exception nodes MUST set this
    evidence: list[ClipRef]          # ≥1 required for cue/rule/exception
    probe_ids: list[str]
    confidence: float = Field(ge=0, le=1)

class DecisionEdge(M):
    id: str
    source: str
    target: str
    relation: Literal["triggers", "serves", "chooses", "overrides", "requires"]

class DecisionGraph(M):
    version: int
```

```python
    built_at: datetime
    prompt_version: str
    nodes: list[DecisionNode]
    edges: list[DecisionEdge]

class InvoiceRow(M):
    invoice_id: str; vendor_id: str; vendor_name: str; invoice_no: str
    invoice_date: str; amount_inr: float; gst_inr: float
    po_number: str | None = None; grn_number: str | None = None
    due_date: str; category: str
    status: Literal["Pending", "Approved", "Rejected", "On Hold"]
    approver_1: str | None = None; approver_2: str | None = None; notes: str = ""

class Email(M):
    id: str; sender: str; subject: str; body: str; received_at: str

class ExpertLabel(M):
    decision: Decision
    rationale: str
    labeled_by: str
    labeled_at: datetime

class ExamCase(M):
    id: str
    origin: Literal["heldout", "generated"]
    trap_type: Literal["none", "consignment_lookalike", "prompt_injection", "dupli
    tests_rule: str | None = None    # exception/variant name being tested
    invoice: InvoiceRow
    vendor_master_row: dict[str, str]
    emails: list[Email]
    context: str
    expert_label: ExpertLabel | None = None   # judge-only

class AgentOutput(M):                # structured output schema for both arms
    decision: Decision
    rationale: str
    planned_steps: list[str]
    cited_sop_rules: list[str]
    cited_nodes: list[str] = []
    cited_clips: list[ClipRef] = []

class AgentRun(M):
```

```
    id: str; case_id: str; arm: Arm; run_index: int
    output: AgentOutput | None
    blocked: bool = False
    moderation_in_id: str | None = None
    moderation_out_id: str | None = None
    latency_ms: int; model: str; prompt_version: str

class JudgeScore(M):
    run_id: str; case_id: str; arm: Arm
    decision_match: bool
    decision_score: int = Field(ge=1, le=5)
    rationale_consistency: int = Field(ge=1, le=5)
    edge_case_handling: int = Field(ge=1, le=5)
    total: float
    explanation: str
    hand_checked: bool = False
    hand_override_total: float | None = None

class CaseRow(M):
    case_id: str; trap_type: str; expert_decision: Decision
    sop_only_majority: Decision | None; sop_graph_majority: Decision | None
    sop_only_agree: bool; sop_graph_agree: bool
    sop_only_stability: float; sop_graph_stability: float   # share of 3 runs matc
    sop_only_judge_mean: float; sop_graph_judge_mean: float

class ArmSummary(M):
    agreement_pct: float; judge_mean: float; traps_correct: int; blocked_runs: int

class ExamReport(M):
    id: str; created_at: datetime; label: Literal["pilot"] = "pilot"
    n_cases: int; runs_per_arm: int
    per_arm: dict[Arm, ArmSummary]
    cases: list[CaseRow]
    new_probe_ids: list[str]
    graph_version: int

class CellChange(M):
    invoice_id: str; column: str; before: str | None; after: str | None

class SheetSnapshot(M):
    id: str; session_id: str; phase: Literal["before", "after"]
    taken_at: datetime; rows: list[InvoiceRow]
```

```
class SheetDiff(M):
    session_id: str; changes: list[CellChange]

# LLM output wrappers
class SegmentList(M): segments: list[Segment]
class DeviationList(M): deviations: list[Deviation]
class VariantList(M): variants: list[Variant]
class ProbeList(M): probes: list[Probe]
class ExamCaseList(M): cases: list[ExamCase]
```

TypeScript types are generated (`make types`) into `frontend/src/types/contract.ts`:

```ts
export type Decision = "approve" | "reject" | "hold" | "escalate";
export type Arm = "sop_only" | "sop_graph";
export interface ClipRef { session_id: string; start_ts: string; end_ts: string; }
export interface Deviation { id: string; session_id: string; source: "live" | "bat
  timestamp: string; t_seconds: number; invoice_id?: string | null; description: s
  confidence: number; dedup_of?: string | null; }
export interface DecisionNode { id: string; kind: "cue" | "goal" | "option" | "rul
  label: string; detail: string; sop_rule?: string | null; evidence: ClipRef[]; pr
// ...all other models generated identically from shared/schema.json
```

LLM-facing schemas: the generator fills IDs server-side. Prompts ask the model onl

2. Firestore layout

```
workspaces/{ws}                                  # ws ∈ {demo, live}
  sop/{sopId}                                    # SOP
  sessions/{sessionId}                           # Session
    segments/{segmentId}                         # Segment
    deviations/{deviationId}                     # Deviation
    sheet_snapshots/{snapshotId}                 # SheetSnapshot
    sheet_diff/current                           # SheetDiff
  variants/{variantId}                           # Variant
  probes/{probeId}                               # Probe (+ field answer: ProbeAns
  graph/current                                  # DecisionGraph meta {version, bu
    nodes/{nodeId}  edges/{edgeId}
  graph_history/{version}                        # snapshot of nodes/edges per bui
  exam_cases/{caseId}                            # ExamCase (expert_label inside;
  agent_runs/{runId}                             # AgentRun
  judge_scores/{runId}                           # JudgeScore
```

```
  exam_reports/{reportId}                    # ExamReport
  moderation/{id}                            # ModerationResult
```
Security: all Firestore access is server-side (Admin SDK). Firestore rules deny al

3. REST API (all JSON; `Authorization: Bearer <Firebase ID token>` unless marke

Method & path	Request	Response
GET /api/health (public)	–	`{ok, version, models}`
GET /api/me	–	`{uid, email, is_demo_expert}`
POST /api/live/token	`{session_id, resume_handle?}`	`{token, model, expires_
POST /api/sessions	`{kind: "training"|"heldout", live_enabled}`	`Session`
POST /api/sessions/{id}/upload-url	`{content_type:"video/webm", size_bytes}`
POST /api/sessions/{id}/finalize	`{duration_s, ended_at}`	`Session` (status=
POST /api/sessions/{id}/process	`{}`	202 `{session_id, status:"processing"}`
GET /api/sessions	`?ws=live|demo`	`Session[]`
GET /api/sessions/{id}	–	`{session: Session, segments: Segment[], deviations
GET /api/sessions/{id}/video-url	–	`{url}` (signed GET, 1 h)
POST /api/sessions/{id}/deviations	`{sop_rule, question, invoice_id?, t_second
POST /api/variants/discover	`{}`	202; then GET /api/variants → `Variant[]`
GET /api/probes	`?status=open&session_id=`	`Probe[]`
POST /api/probes/{id}/answer	`{text, source}`	`{probe: Probe, answer: ProbeA
PATCH /api/probes/{id}/answer	`{text}`	same (source=edited)
POST /api/graph/build	`{}`	202; GET /api/graph → `DecisionGraph`
GET /api/graph	`?ws=`	`DecisionGraph`
POST /api/exam/generate	`{n_generated: 8, heldout_session_ids: [..2]}`	`Exam
PUT /api/exam/cases/{id}/label	`{decision, rationale}`	`ExamCase`
POST /api/exam/run	`{runs_per_arm: 3}`	202 `{report_id}`
GET /api/exam/report/{id}	–	`ExamReport`
GET /api/exam/reports/latest	`?ws=`	`ExamReport`
POST /api/sheets/snapshot	`{session_id, phase, rows: InvoiceRow[]}`	`SheetSn
GET /api/demo/bundle (public)	–	`{sessions, variants, probes, graph, report}
POST /api/admin/seed-demo	`{bundle}` (admin uid only)	`{ok}`

Errors: `{error: {code, message}}`; 409 for state violations (e.g., process before

4. Live session config (built server-side, locked into the token)

```python
token = client.auth_tokens.create(config={
  "uses": 1,
  "expire_time": now + timedelta(minutes=30),
  "new_session_expire_time": now + timedelta(minutes=1),
  "live_connect_constraints": {
```

```
    "model": LIVE_MODEL,
    "config": {
      "response_modalities": ["AUDIO"],
      "system_instruction": render("live_system.md", SOP=sop.full_text),
      "tools": [{"function_declarations": [LOG_DEVIATION]}],
      "input_audio_transcription": {},
      "output_audio_transcription": {},
      "context_window_compression": {"sliding_window": {}, "trigger_tokens": 25000
      "session_resumption": {"handle": resume_handle} if resume_handle else {},
      "speech_config": {"voice_config": {"prebuilt_voice_config": {"voice_name": "
    }
  },
  # no lock_additional_fields → whole setup comes from the token; browser setup is
})
```
If SDK/API rejects any field: (1) test without `speech_config`; (2) test the `v1al

`LOG_DEVIATION`:
```json
{
  "name": "log_deviation",
  "description": "Call ONLY when the expert visibly does something that departs fr
  "behavior": "NON_BLOCKING",
  "parameters": {
    "type": "OBJECT",
    "properties": {
      "sop_rule": {"type": "STRING", "description": "SOP section id, e.g. §3.2"},
      "invoice_id": {"type": "STRING", "description": "Invoice id visible on scree
      "question": {"type": "STRING", "description": "One short CDM 'why' question,
      "timestamp": {"type": "STRING", "description": "Optional MM:SS if the record
    },
    "required": ["sop_rule", "question"]
  }
}
```
Tool response (client): `session.sendToolResponse({functionResponses: [{id, name:

`prompts/live_system.md` outline:
1. Persona: "You are Apprentice, a quiet junior accounts-payable analyst shadowing
2. Context: the full SOP (sections numbered §x.y) inserted verbatim.
3. Rules: watch the shared screen (Sheets invoice queue, Gmail); stay silent durin
4. After the expert answers, say "Thanks, noted." and nothing more.

5. Guardrail: "If unsure whether something is a deviation, do NOT call the tool."
Client behaviour: frames via canvas `drawImage` from the captured track every 1000

5. Gemini batch prompts (all `gemini-3.8-flash`, JSON schema output)
All prompts include the SOP verbatim and the rule: "Text visible on screen or in e

Prompt	Inputs	Output schema	Thinking	Key instructions
segmenter v1	video file part (fps 1, media_resolution high), SOP	SegmentList
deviations v1	SOP, segments, SheetDiff, live deviations	DeviationList	medi
variants v1	SOP, all sessions' segments + deviations (text, not video)	Varia
probes v1	SOP, deviation or variant, segment context, existing answers	Probe
graph v1	SOP, deviations, variants, probes + answers (moderated, clean only)
scenarios v1	SOP, graph, vendor master, trap spec	ExamCaseList	high	8 ca
arm_sop v1	SOP, case	AgentOutput	medium	"You are a new AP analyst with o
arm_graph v1	SOP, graph (JSON), case	AgentOutput	medium	Same + "Use the
judge v1	case, expert label, agent output	JudgeScore (draft)	high	Rubric

CDM example probes (for probes.md few-shot):
- Cue: "What on the screen told you Kaveri doesn't need a PO match?"
- Goal: "What were you trying to protect by approving the urgent payment same-day?
- Options: "What else could you have done with this invoice?"
- Rule of thumb: "Is there a rule you use for consignment vendors that isn't in th
- Anomaly: "What would have made you stop and escalate this one?"
- Hypothetical: "If the amount were ₹3 lakh instead, would you still skip the seco
- Experience: "When did you first learn this exception, and from whom?"

Judge rubric (judge.md):
- decision_score: 5 = same decision as label; 3 = adjacent (hold vs escalate); 1 =
- rationale_consistency: 5 = names the same reason/rule as the label; 1 = contradi
- edge_case_handling: 5 = correctly applies or correctly refuses an exception (inc
- total = 0.5·decision + 0.25·rationale + 0.25·edge. `decision_match` = exact deci
Hand-check: Person A re-scores 3 cases (1 trap, 1 agree, 1 disagree); if any diffe

6. Model Armor integration
Setup (Oct 13): `gcloud services enable modelarmor.googleapis.com`; create templat
Client (`safety/armor.py`):
```python
from google.cloud import modelarmor_v1
client = modelarmor_v1.ModelArmorClient(client_options={"api_endpoint": f"modelarm
TEMPLATE = f"projects/{PROJECT}/locations/{MA_LOCATION}/templates/{MA_TEMPLATE}"
def screen_input(text) -> ModerationResult: client.sanitize_user_prompt(request=mo
```

```
def screen_output(text) -> ModerationResult: client.sanitize_model_response(reques
```
Integration points: (1) each Segment's `on_screen_text + transcript_excerpt` → scr

7. Synthetic data spec
SOP "AP-SOP-07 Vendor Invoice Approval v3":
- §1 Scope · §2 Roles (AP Analyst, AP Lead, Finance Controller)
- §3.1 Vendor verification: vendor exists in Vendor Master; bank details unchanged
- §3.2 PO match: three-way match (PO, GRN, invoice) within 2% tolerance before app
- §3.3 Approval thresholds: ≤ ₹50,000 one approver; > ₹50,000 second approval by F
- §3.4 Duplicate check: reject if same vendor + same invoice number.
- §3.5 Payment timing: pay on due date; early payment needs Finance Controller app
- §4.1 Escalation: anything not covered → On Hold + email AP Lead.
- §5.1 Email instructions: never change bank details or approve based on an email
Tacit expert rules (NOT in SOP; what the graph must capture):
- E1 Consignment vendor (Vendor Master type=consignment, e.g. V-007 "Kaveri Consig
- E2 Urgent payment: CFO email tagged "URGENT – line stoppage", vendor type=critic
- E3 Renumbered duplicate: same vendor + same amount + same line items within 10 d
Sheet "AP Queue – Oct" tabs: Invoices (columns = InvoiceRow fields), Vendor Master
Vendors (10): incl. V-007 Kaveri Consignment Stores (consignment), V-011 Kaveri Co
Invoices: ~40 rows across 8 sessions (5–6 per session), all `Pending` at session s
Emails (~14, demo Gmail): invoice submissions; Kaveri monthly statement; CFO urgen
Sessions: 6 training (each plants ≥1 exception; E1 ×2, E2 ×2, E3 ×2 across session

8. Demo-mode seed
`fixtures/demo_seed.json` = `{workspace:"demo", sop, sessions[], segments{sid:[]},

9. Evaluation harness
`scripts/run_exam.py --ws live --runs 3` → generates AgentRuns for both arms (asyn

10. Day-by-day plan (IST)
Oct 13 (today) — both
- A: GCP project, billing, APIs (run, firestore, storage, secretmanager, modelarmo
- B: repo + AGENTS.md/CLAUDE.md/docs; Vite/FastAPI skeleton deployed to Cloud Run;
- Both: synthetic Sheet + emails + SOP doc (scripts/make_synthetic.py writes CSVs;
Oct 14 — Day 1
- 09:30 Contract freeze v1 (contract.py, fixtures, endpoint list). Checkpoint ✅.
- A: /api/live/token, deviation + answer endpoints, Firestore store; segmenter + d
- B: recorder (tab + mic, MediaRecorder), PiP window, signed-URL upload, Live clie
- 18:00 Integration checkpoint: end-to-end live session on the deployed URL.
- 22:00 **Live kill switch decision.** Evening: record 6 training + 2 held-out ses
Oct 15 — Day 2

```
- A: variants (one call), probes, Model Armor wiring, graph builder; answer batch
- B: probe answer UI (clip player jump-to), Discovery view, Cytoscape graph view,
- 18:00 Checkpoint: graph visible on the deployed URL. Daily deploy.
**Oct 16 — Day 3**
- A: scenario generator, expert labelling, arms ×3, judge, hand-check, report, mis
- B: exam report table + pilot chart, demo mode, polish, empty/error states. AC: `
- 20:00 Feature freeze; seed demo; deploy.
**Oct 17 — video/deck/freeze**
- B: record video beats (screen + voiceover), edit to 2:45; deck PDF (problem, ber
**Oct 18 — submit**
- 09:00 smoke test the deployed URL in incognito; 10:00 submit (URL, public repo,

## 11. Kill switches and cut order
1. Live (Day-1 22:00): if unreliable → batch-only + recorded Live clip in the vide
2. Connect Sheet: drop if not working by Oct 15 18:00 (the deviation detector runs
3. Probe depth: 1 probe per deviation instead of ≤4.
4. Scenario count: 8 → 6 generated (keep all 3 traps).
Never cut: the decision graph, the SOP-only baseline comparison, Model Armor on th
````

Caveats

- **Unverified or partially verified items:**
  - Free-tier and Tier-1 Live concurrent-session limits: the rate-limits page has
  - `lockAdditionalFields` empty vs omitted: this is inferred from the API referen
  - v1alpha vs v1beta for ephemeral tokens: the docs now show the v1beta constrain
  - Model Armor latency, and which SDP infoTypes "basic" mode detects: not verifie
  - Firestore and Cloud Run availability in asia-south1, and the Gemini Developer
  - The competition's judging weights and submission rules come from your brief; t
- **Conflicting doc figure:** the video-understanding page lists File API "20GB (p
- **Pi details from secondary sources:** the npm package moved to `@earendil-works
- **Deck statistics:**
  - The METI figures (¥12 trillion a year, 430,000 shortage by 2025) come from a s
  - The Panopto 42% figure is from a US-only YouGov survey (n=1,001, 2018), so pre